

ENCODE instruction → hex: write it, copy each part above its field, look up bits, read hex

\$t	t0-t7	0	1	d	d	d + 3 bits			
	t8-t9	1	1	0	0	d	d + 0 (t8) / 1 (t9)		
\$s	s0-s7	1	0	d	d	d + 3 bits			
\$a	a0-a3	0	0	1	d	d + 2 bits			
\$v	v0-v1	0	0	0	1	d	d + 1 bit		
\$k	k0-k1	1	1	0	1	d	d + 1 bit		
d	0	1	2	3	4	5	6	7	
bits	000	001	010	011	100	101	110	111	

no pattern	
\$zero	00000
\$at	00001
\$gp	11100
\$sp	11101
\$fp	11110
\$ra	11111

type	instr	code	starts	type	instr	code	starts
R	add	fn 100000	00-03	J	jal	op 000011	0C-0F
R	sub	fn 100010	00-03	I	beq	op 000100	10-13
R	and	fn 100100	00-03	I	bne	op 000101	14-17
R	or	fn 100101	00-03	I	addi	op 001000	20-23
R	nor	fn 100111	00-03	I	slti	op 001010	28-2B
R	slt	fn 101010	00-03	I	andi	op 001100	30-33
R	sll	fn 000000	00-03	I	ori	op 001101	34-37
R	srl	fn 000010	00-03	I	lui	op 001111	3C
R	jr	fn 001000	00-03	I	lw	op 100011	8C-8F
J	j	op 000010	08-0B	I	sw	op 101011	AC-AF

0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

fx-97SG X: MENU Base-N icon □. DEC = □² HEX = □[■] BIN = log A (-) B °/'' C x⁻¹ D sin E cos F tan. Base key after □ converts the answer. Negatives: -3 = FFFFFFFD. BIN shows all 32 bits, 2 lines of 16.

R-type

add sub and or nor slt

instr

\$

,

\$

,

\$

rd

rs

rt

bits go rs, rt, rd

op

rs

rt

rd

shamt

funct

0 0 0 0

0 0

0 0 0

0 0 1 0

0x

0

2

funct: add100000 sub100010 and100100 or100101 nor100111 slt101010. jr \$ra = 0x03E00008

R-type

shifts: sll srl

instr

\$

,

\$

,

rd

rt

shamt

rs = 0; source is rt

op

rs

rt

rd

shamt

funct

0 0 0 0

0 0 0 0

0 0 0

0 0 0

0 0 0 0

0x

0

0

funct: sll000000 srl000010. shamt = 0-31 in 5 bits.

I-type

addi slti andi ori

instr

\$

,

\$

,

rt

rs

imm

bits go rs, rt

op

rs

rt

imm

0 0 1

0x

op: addi001000 slti001010 andi001100 ori001101. andi/ori: imm unsigned.

Negative imm: 65536 - |imm| = = 0x.

lui rt, imm: op 001111, rs = 0, word 3C0/3C1...

I-type

lw / sw

instr

\$

,

(\$

)

rt

imm

rs

bracket register is rs

op

rs

rt

imm

1 0 0 0

1 1

0x

op: lw100011 sw101011. Negative offset: 65536 - |imm| = = 0x.

Calc: BIN op rs rt (16 bits) □ HEX → first 4 digits. imm: DEC 0 □ 12 □ HEX → FFFFFFFF4, keep last 4

I-type

branch encode: beq bne

instr

\$

,

\$

,

rs

rt

label

no swap: rs, rt

op

rs

rt

imm

0 0 0 1

0

0x

1

op: beq000100 bne000101. target 0x - (PC+4) 0x = ÷ 4 =

Backward (imm < 0): 65536 - |imm| = = 0x.

Calc: HEX □ target □ PC □ 4 □ ÷ 4 □ → keep last 4 digits (FFFFFFFD → FFFD)

J-type

jump encode: j jal

count instructions, not bytes

1 start address

0x

drop, ÷ 4

0x

2 ± instr. count

± (dec) = (hex)

0x

1st digit must be 0-3

3 + opcode

0x

0

← +08000000 j / +0C000000 jal

Label given as an address? Use it in row 1 with 0 in row 2. Label before the start? Subtract. Label's 1st digit must match PC+4's 1st digit.

e.g. il at 00400020, label = i12: 0400020 ÷ 4 = 0100008, +11 = +B → 0100013 → 08100013

Calc: HEX target □ 4 □ 8000000 □ → word (jal: □ C000000). Drop target's 1st digit first if not 0.

DECODE

hex → instruction: find the row by the first 2 hex digits, fill hex, bits, fields, then follow the lines up

\$t	t0-t7	0	1	d	d	d	+ 3 bits		
	t8-t9	1	1	0	0	d	+ 0 (t8) / 1 (t9)		
\$s	s0-s7	1	0	d	d	d	+ 3 bits		
\$a	a0-a3	0	0	1	d	d	+ 2 bits		
\$v	v0-v1	0	0	0	1	d	+ 1 bit		
\$k	k0-k1	1	1	0	1	d	+ 1 bit		
d	0	1	2	3	4	5	6	7	
Bits	000	001	010	011	100	101	110	111	

no pattern	
\$zero	00000
\$at	00001
\$gp	11100
\$sp	11101
\$fp	11110
\$ra	11111

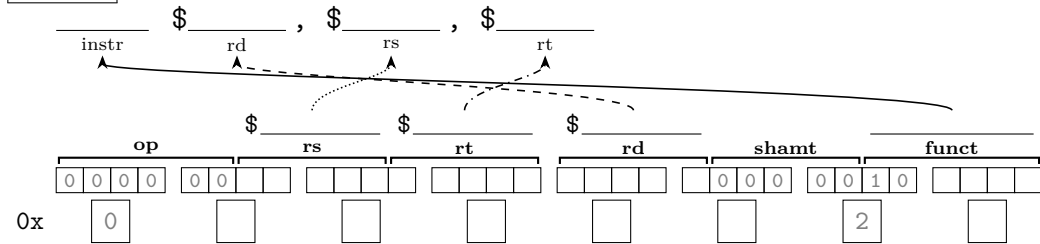
type	instr	code	starts	type	instr	code	starts
R	add	fn 100000	00-03	J	jal	op 000011	0C-0F
R	sub	fn 100010	00-03	I	beq	op 000100	10-13
R	and	fn 100100	00-03	I	bne	op 000101	14-17
R	or	fn 100101	00-03	I	addi	op 001000	20-23
R	nor	fn 100111	00-03	I	slli	op 001010	28-2B
R	sllt	fn 101010	00-03	I	andi	op 001100	30-33
R	sll	fn 000000	00-03	I	ori	op 001101	34-37
R	srl	fn 000010	00-03	I	lui	op 001111	3C
R	jr	fn 001000	00-03	I	lw	op 100011	8C-8F
J	j	op 000010	08-0B	I	sw	op 101011	AC-AF

0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

fx-97SG X: MENU Base-N icon □. DEC = x^2 HEX = $x^{\blacksquare}$ BIN = log A (—) B □''' C x^{-1} D sin E cos F tan. Base key after □ converts the answer. Negatives: $-3 = \text{FFFFFFFD}$. BIN shows all 32 bits, 2 lines of 16.

R-type	add sub and or nor slt jr, decode: starts 00–03
--------	---

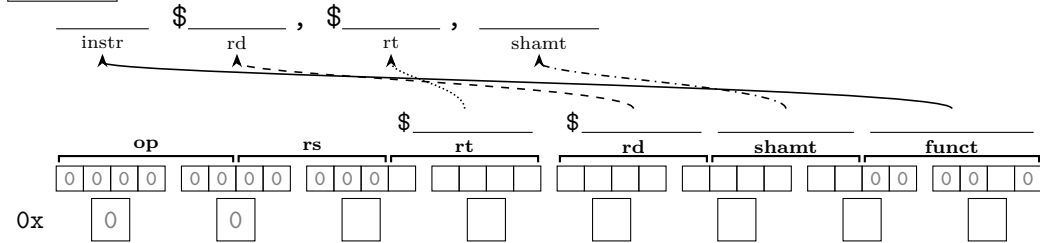
fields read rs, rt, rd



Given 0x_____, funct: add100000 sub100010 and100100 or100101 nor100111 slt101010 jr
001000 (only rs)

R-type	shift sll srl, decode: funct 00 / 02
--------	--------------------------------------

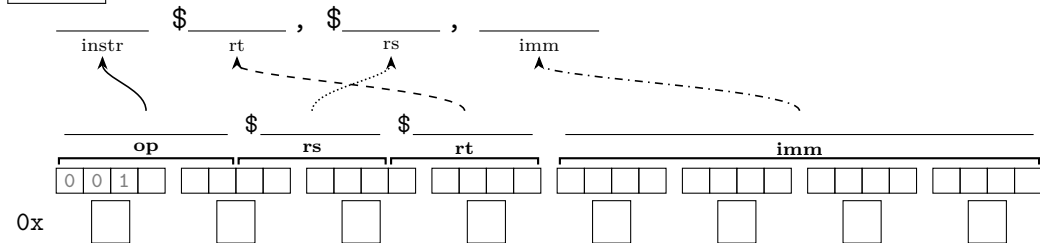
source is rt



Given 0x_____. funct: sll 000000 srl 000010. shamt in decimal = shift amount.

I-type	addi slti andi ori lui, decode: starts 20–23, 28–2B, 30–37, 3C
--------	--

fields read rs, rt

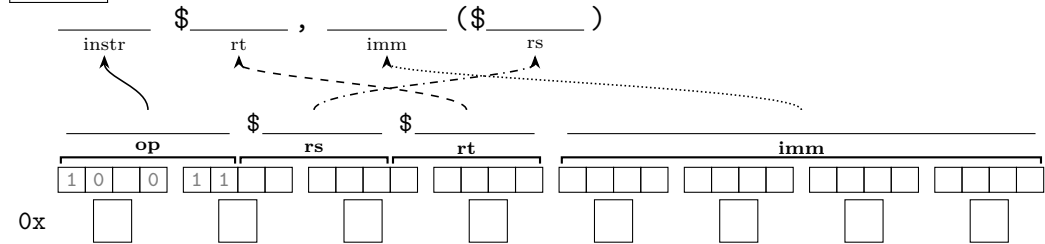


Given 0x_____. op: addi 001000 slti 001010 andi 001100 ori 001101 lui 001111

addi/slti: imm $\geq$ 0x8000? signed = imm - 65536 = _____

I-type	lw / sw, decode: starts 8C–8F / AC–AF
--------	---------------------------------------

rs goes in the brackets

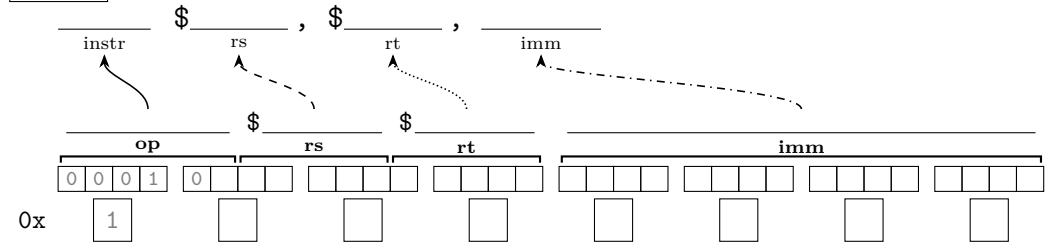


Given 0x_____. op: lw10011 sw101011. imm $\geq$ 0x8000? signed = imm - 65536 = _____

Calc: **HEX** first 4 digits $\equiv$ **BIN** $\rightarrow$ bottom line: split 6 | 5 | 5. imm $\geq$ 8000: imm $\boxed{-}$ 10000 $\equiv$ **DEC** $\rightarrow$ signed

I-type	branch beq bne, decode: starts 10–17
--------	--------------------------------------

imm = 0: next instruction



Given 0x_____. PC 0x_____

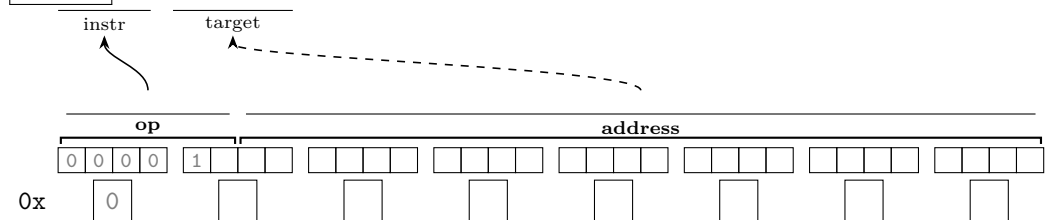
op: beq 000100 bne 000101. imm $\geq 0x8000$? signed = imm - 65536 = _____ $\times 4$ = _____ + PC+4 = _____

target 0x_____

Calc: `HEX PC + 4 + (imm - 10000) * 4 = target`. `imm < 8000`: leave out the `-10000`

J-type	jump j jal, decode: starts 08–0F
---------------	----------------------------------

PC+4, not PC



Given 0x_____. PC 0x_____

$$\text{address} = \text{word} - 08000000 / 0C000000 = 0x______ \times 4 = 0x______ \quad (7 \text{ digits})$$

target = 0x [1st digit of PC+4] [7 digits] = 0x_____

Calc: `HEX` `(` word `-` 8000000 `)` `*` 4 `=` $\rightarrow$ 7 digits (jal: - C000000); PC+4's 1st digit in front